Business 41204

Machine Learning

Introduction to Foundation Models
(i.e., LLMs)

Outline:

1. Introduction
2. Anatomy of an LLM: Tokens and Vector Representations
3. Anatomy of an LLM: Attention and Transformers
4. Anatomy of an LLM: Next Token Prediction
5. Few-Shot Learning
6. Agents

1. Introduction

Foundation Models

Classic ML (and AI) paradigm – build a separate model for each task

- Basically what we've done in this class
- Can be very powerful and "easy" to explain why it should work
- **BUT**, bespoke – basically start from scratch any time task changes
  - human capital helps
  - Transfer learning (as we talked about in DNN section) might give a "warm start"

New AI Approach: Foundation Models

- Train one **very large model** on massive, diverse data
- Then adapt to many downstream tasks
  - Initially text – moving to multimodal data (text, images, sounds, …) as the vast majority of human data/experience is not text
  - The LLMs we all know and love live here
- Think about foundation models as digital "ant brains"

Term coined in "On the Opportunities and Risks of Foundation Models" (Bommasani et al., 2021)

Why Foundation Models?

Scale seems to generate emergent behavior beyond initial training task

- Training on huge, diverse data leads to capabilities across many domains including domains not specifically trained for
  - E.g. models trained *only* to predict next token exhibit comprehension of language, ability to do simple reasoning, knowledge of general concepts, …

Means one model can be applied/adapted to many tasks

- E.g. use ChatGPT/Claude/DeepSeek for
  - Customer support automation
  - Marketing content generation
  - Document analysis and summarization
  - Coding copilots …

Foundation models offer a potential "General purpose technology"

- another, much older concept referred to as a GPT ☺ (e.g. Bresnahan and Trajtenberg (1995) "General purpose technologies 'Engines of growth'?")

2. Anatomy of an LLM

Tokens and Vector Representations

Tokens

LLMs don’t “see” words – they see tokens

- token = integer ID corresponding to learned subword
  - e.g., word, character(s), punctuation, whitespace

E.g. “Prof. Hansen teaches machine learning to MBA students.”

Tokenization:

- Token IDs: 19117, 13, 88786, 45459, 7342, 7524, 316, 53298, 4501, 13
- Token strings: 'Prof', '.', ' Hansen', ' teaches', ' machine', ' learning', ' to', ' MBA', ' students', '.'

Seems pretty natural. Let’s look at slightly more complicated sentence.

Tokens

First sentence in Tolkien's *The Fellowship of the Ring*:

> "*When Mr. Bilbo Baggins of Bag End announced that he would shortly be celebrating his eleventy-first birthday with a party of special magnificence, there was much talk and excitement in Hobbiton.*"

Tokenization:

- Token IDs: 5958, 9655, 13, 30007, 1692, 418, 14013, 1564, 328, 26231, …
- Token strings: 'When', ' Mr', '.', ' Bil', 'bo', ' B', 'agg', 'ins', ' of', ' Bag', …, ' there', ' was', ' much', ' talk', ' and', ' excitement', ' in', ' Hobbit', 'on', '.'

Note that we now get single characters (e.g., "B") and small sets of characters (e.g., "Bil")

- Important for success of models
- Keeping track of literally all words would lead to too many features
  - Rare words, typos, …
- Could do everything with just single characters, but also inefficient
- Older GPT style models (e.g., GPT-3) use on the order of 50K tokens

Vector Representation

We're not done – assigning just an integer to each token makes "meaning and math" awkward

- How do I think about how "close" token 30007 on the previous slide ("Bil") is to token 418 ("B")?
- Integer assigned to a token is rather arbitrary

Now map tokens to lists of real numbers (vectors)

- That is, we "embed" the tokens in a ***learned*** vector space
  - Features of the embedding learned with other features of the LLM by using data
  - Vector space has well-defined addition and multiplication operations
    - LLM embeddings generally live in normed vector spaces so meaningful definition of distance
    - Similarity often measured with "cosine similarity" (think correlation)
  - I've seen embedding dimensions ranging from ≈ 1000 to ≈ 10000

A "Classic Demo"

A common demo at this point is to consider embeddings for

- good, bad, happy, sad
  - e.g., v_{good} = (-0.009, 0.000, -0.011, 0.057, -0.021, -0.017, -0.022, -0.051, 0.016, 0.005, …)
  - e.g., v_{bad} = (0.019, -0.018, -0.055, 0.073, 0.046, -0.033, 0.008, 0.032, 0.007, -0.023, …)
    - 1536 dimension vectors in our example
  - These are just lists of numbers we can do arithmetic with
- Can create new "word" = $v_{good} + (v_{sad} - v_{bad})$

Similarity				
	v_{bad}	v_{happy}	v_{sad}	$v_{good} + (v_{sad} - v_{bad})$
v_{good}	0.612	0.662	0.445	0.694

Certainly don't want to overstate, but looks like embeddings are capturing something about the "geometry of meaning"

Contextual Embeddings

LLMs are trained using architectures that capture context in language

- Attention/Transformers: More on this in the next section

Embeddings are contextual embeddings

- Embedding vectors will change depending on the complete input text
- Embeddings pulled from big LLMs (e.g. ChatGPT) are typically returned at the input-text level
  - I.e. you will get one embedding for the phrase “Prof. Hansen likes tacos.” (not a separate embedding per token)
  - The embedding will be different from the one returned from the phrase “Prof. Hansen likes fantasy novels.”
  - These embeddings aggregate up the token embeddings based on the underlying structure

Contextual Embedding Illustration

Here we look at two ways to construct phrase level embeddings:

1. "Bag-of-Words Avg" gets an embedding token by token and then averages to form the phrase level embedding
2. "Contextual Embedding" is the contextual embedding

Cosine similarity between Phrase 1 and Phrase 2 embedding

Phrase 1	Phrase 2	**Bag-of-Words Avg**	**Contextual Embedding**
bank	river bank	0.843	0.432
bank	investment bank	0.843	0.614
bank	bank account	0.857	0.674
bank	piggy bank	0.825	0.526
bank	sat on the river bank	0.691	0.313
river bank	investment bank	0.743	0.386
river bank	bank account	0.747	0.320
river bank	piggy bank	0.755	0.311
river bank	sat on the river bank	0.802	0.619
investment bank	bank account	0.797	0.506
investment bank	piggy bank	0.751	0.493
investment bank	sat on the river bank	0.686	0.280
bank account	piggy bank	0.751	0.524
bank account	sat on the river bank	0.694	0.251
piggy bank	sat on the river bank	0.692	0.267
		All look similar because all have word "bank"	Context helps distinguish phrases

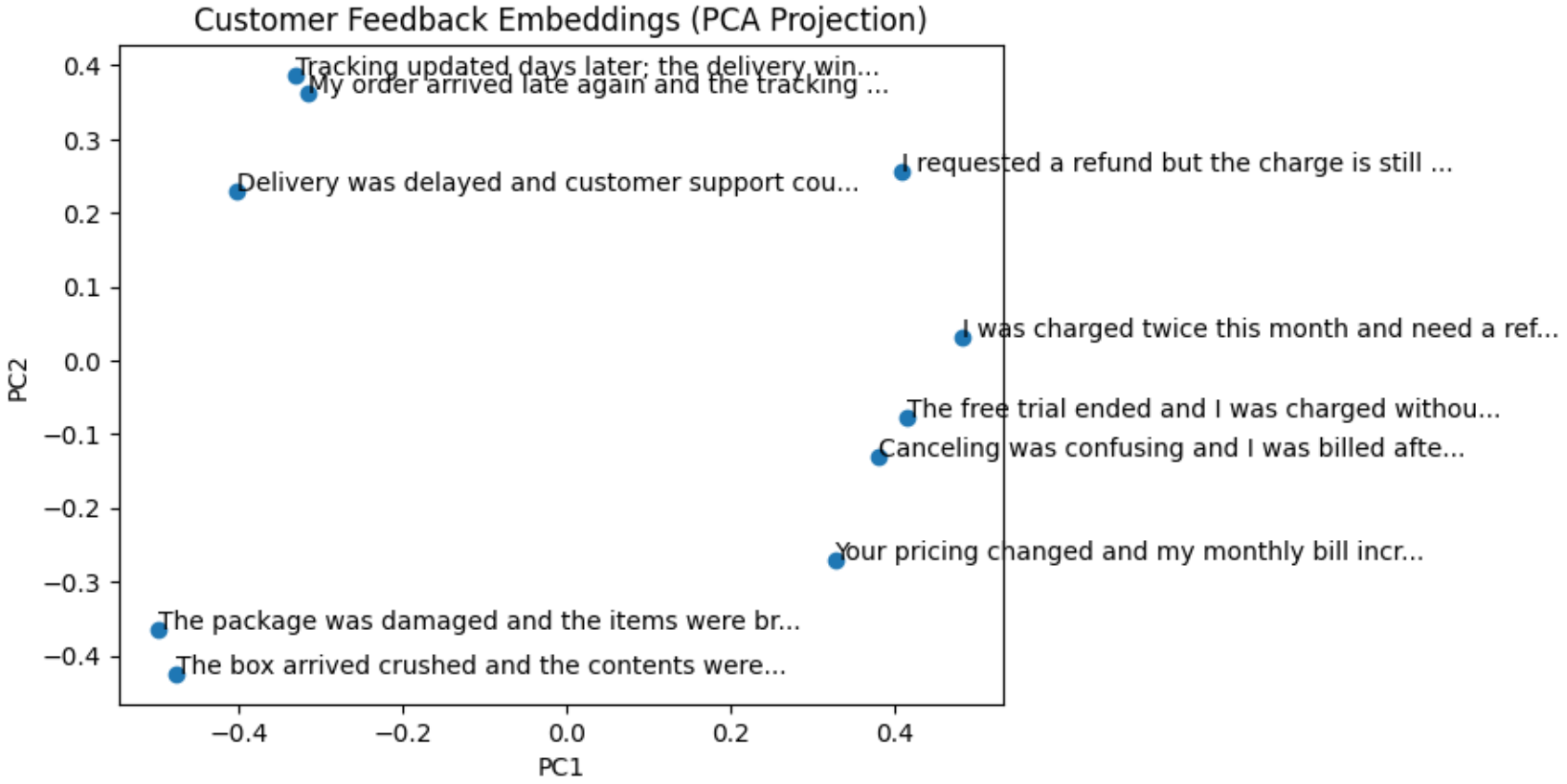
Example: Clustering with Embeddings

Contextual embeddings may be directly useful.

Here, we look at a toy example where we have 10 customer reviews.

- Extract embedding for each review
- Can see semantically similar complaints “close” to each other

Could use for clustering complaint types, routing support tickets, …



3. Anatomy of an LLM

Attention and Transformers

“Words” in Context

Tokens (represented as vectors) provide our basic unit of analysis

- implementations also add a *positional vector* to token vectors to help capture token sequence

Of course, meaning depends on context:

- I take the train to work most days.
- I need to take some time to train for a marathon.

Attention provides a *computationally tractable* way to model context

- going to do some math and abstraction in next few slides, BUT

Basic idea is relatively simple

- when representing a given token, “pay attention” to other tokens that are important for interpretation
- do this by looking at *all other tokens* and “giving high weight” to those that heavily influence meaning

Attention Math 1

Math for attention is actually pretty straightforward

Recall that we have a "sentence" represented by a collection of tokens $(t_1, \dots, t_S)$

Each token is further represented by a vector: $(x_1, \dots, x_S)$

Attention starts by making three ("weighted") copies of each x_j:

- Query vector: $q_j = (q_{j1}, \dots, q_{jR}) = \left(\sum_d w_{1d}^Q x_{jd}, \dots, \sum_d w_{Rd}^Q x_{jd}\right)$
- Key vector: $k_j = (k_{j1}, \dots, k_{jR}) = \left(\sum_d w_{1d}^K x_{jd}, \dots, \sum_d w_{Rd}^K x_{jd}\right)$
- Value vector: $v_j = (v_{j1}, \dots, v_{jH}) = \left(\sum_d w_{1d}^V x_{jd}, \dots, \sum_d w_{Hd}^V x_{jd}\right)$
- weight matrices W^Q, W^K, W^V are parameters to be learned

A big part of the success of attention is that computation is simple – we're just multiplying and adding – doing simple linear algebra:

- X: matrix of all token vectors
- W^Q, W^K, W^V: matrices of query, key, value parameters (stacked conformably with X)
- $Q = XW^Q, K, = XW^K, V = XW^V$: matrices of query, key, and value vectors

Matrix multiplication (multiplying and adding) corresponds to rotation and scaling.

We'll illustrate in a few slides.

Attention Math 2

Each of our initial tokens is now represented by three vectors

- E.g. Input: Grass is green
  - Grass: (q_1, k_1, v_1); is: (q_2, k_2, v_2); green: (q_3, k_3, v_3)

For each token, we now "score" it versus all tokens (including itself)

- score will determine how much focus different tokens get when we encode the given token
- score is taken by calculating a measure of "similarity" between query and key
  - specifically, score between tokens i, j: $s_{ij} = \sum_r q_{ir} k_{jr}$
    - think covariance

All scores obtained from matrix multiplication: QK'

We then normalize the scores

- divide by the square root of the key/query dimension
- normalize everything to be between 0 and 1 and sum to 1
  - by applying softmax function
- gives numbers a_{ij} (such that $\sum_j a_{ij} = 1$) capturing the "weight" to put on token j when "thinking about" token i

Attention Math 3

Finally, we obtain a new representation for each token by applying our weights (the a_{ij}) to the value vector

Vector representation for token i output from *self-attention layer:*

- $z_i = (z_{i1}, \ldots, z_{iH}) = (\sum_j a_{ij} v_{j1}, \ldots, \sum_j a_{ij} v_{jH})$

This representation is a weighted combination of all the tokens where weights depend on "similarity"

- The parameter matrices W^Q, W^K, W^V let us adapt what we mean by similarity by realigning the vectors

Let's look at a (simplified) visualization

The entire process is conveniently represented and calculated using matrix algebra:

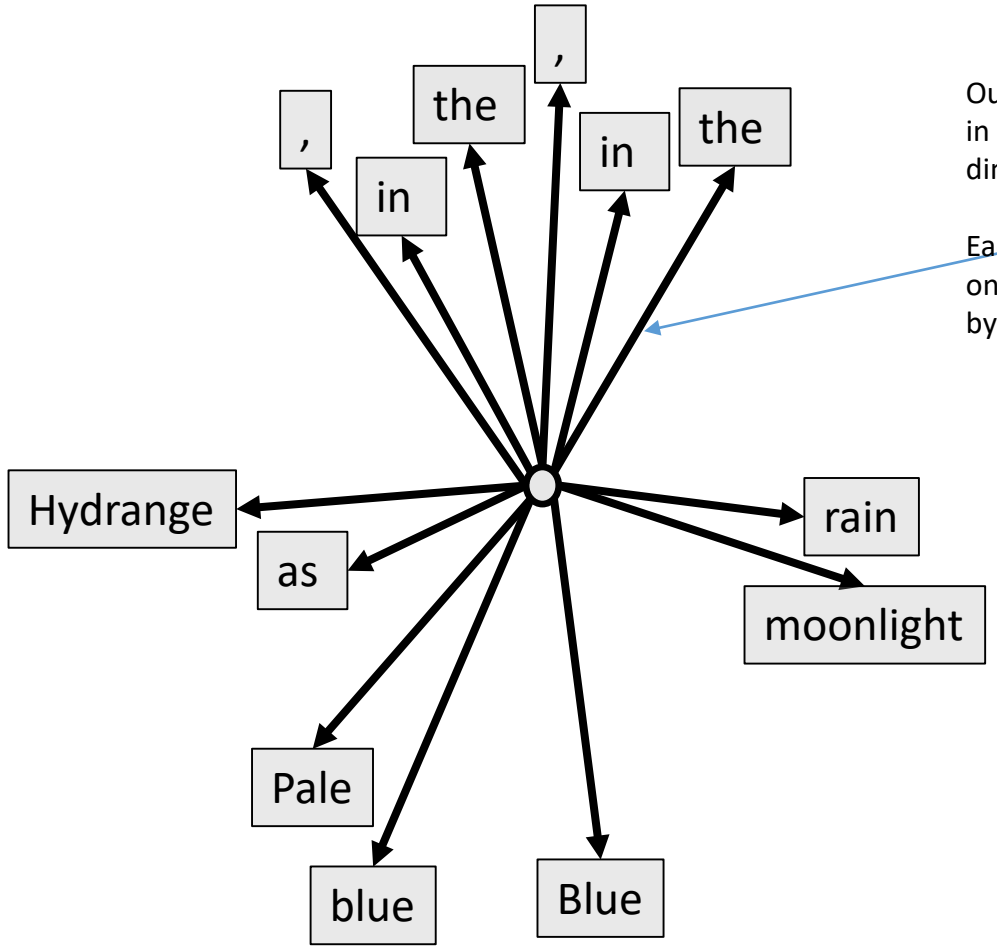
$$Z = \text{softmax}(QK'/\sqrt{R})V$$

Visualizing Attention

#	decoded token
1	`Hydrange`
2	`as`
3	`,`
4	`Pale`
5	`blue`
6	`in`
7	`the`
8	`rain`
9	`,`
10	`Blue`
11	`in`
12	`the`
13	`moonlight`

Hydrangeas | By Masaoka Shiki | Translation by Lucien Stryk | Photo: from Divinity School Café

Stylized Embedding



Our token representation in this example is two-dimensional.

Each token is just a point on a plane represented by an arrow.

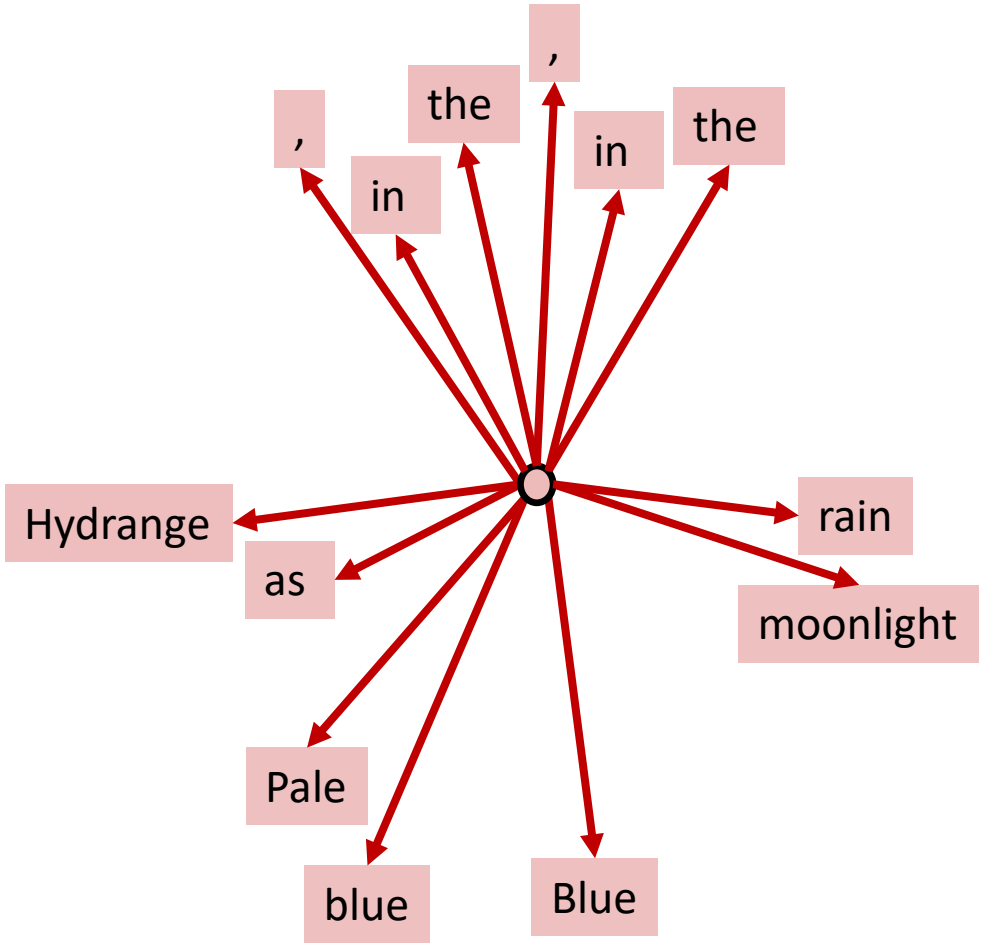
Recall that we are representing tokens as vectors.

Here, our embedding includes information about location in sentence

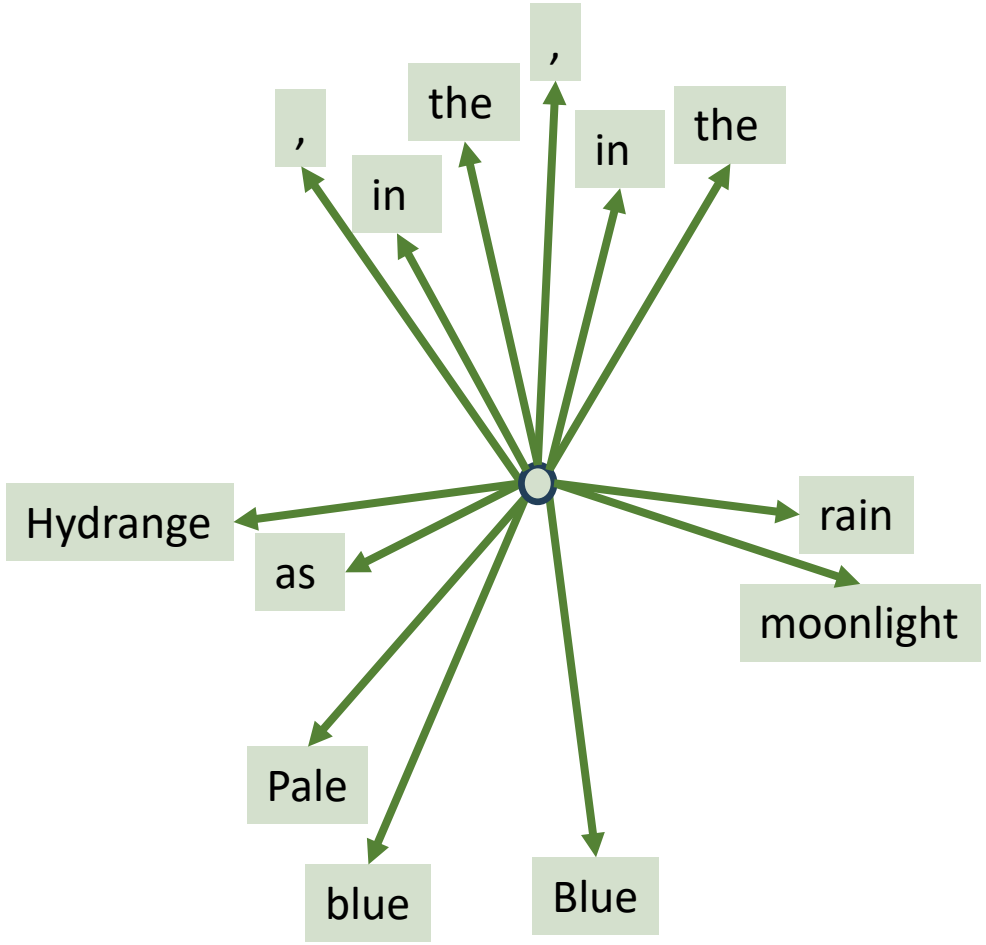
- Earlier positions in sentence appear closer to the left
- Nouns appear near horizon
- Descriptors appear below horizon
- Grammatical tokens appear above horizon

Create Keys and Values

First create two copies of embedded tokens.
Work with 2D representation and color code them for visualization.



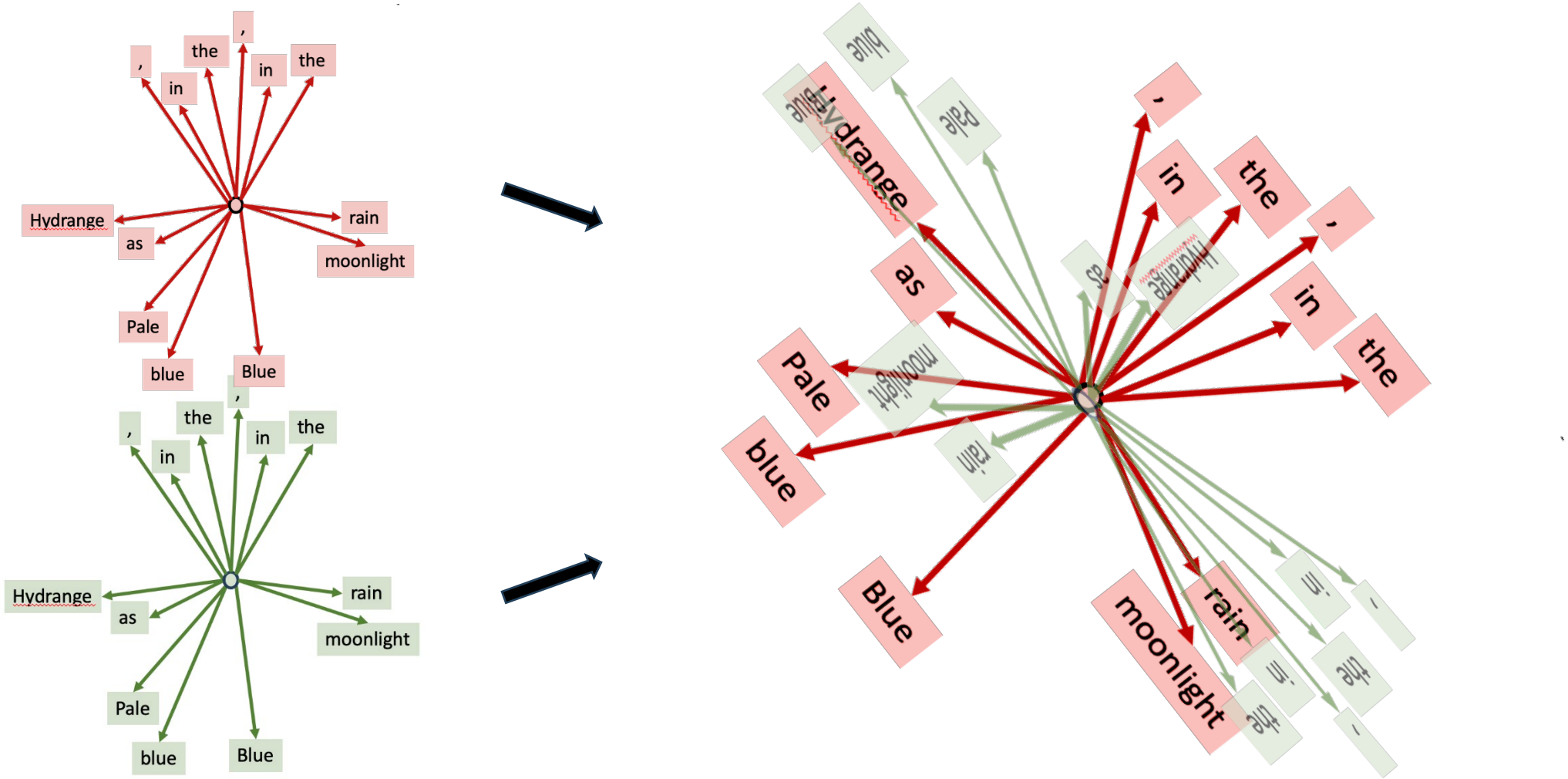
Query copy of tokens



Key copy of tokens

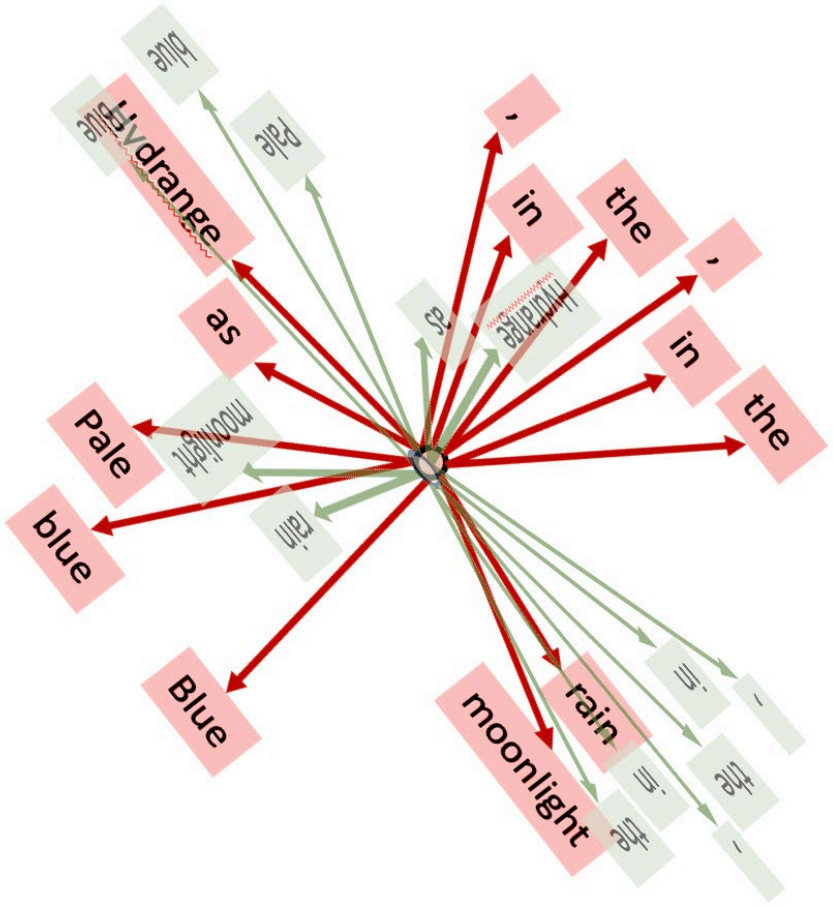
Rotate, scale, overlay, and compare

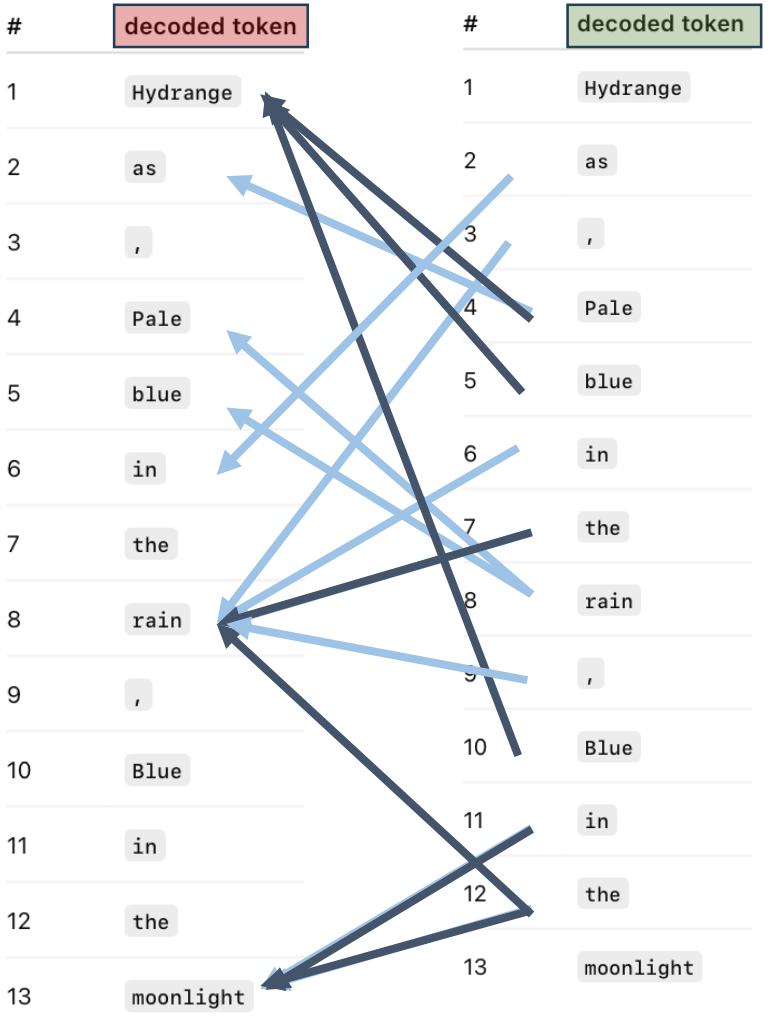
- We multiply embedding coordinates with matrices W^K, W^Q
- Matrix multiplication may rotate, scale, and possibly flip-over the vectors.
  - Geometrically, this is all matrix multiplication does.



Attention Measures Overlap

Dark blue indicates “large query-key similarity”
Light blue indicates “moderate query-key similarity”





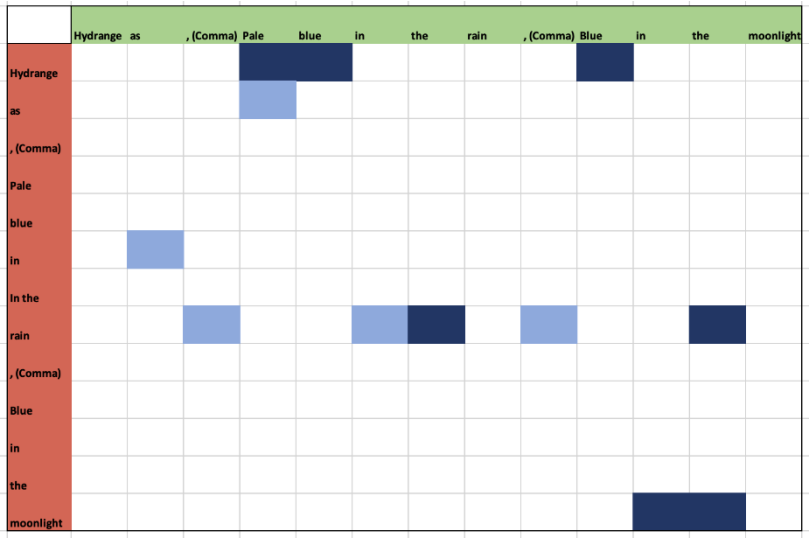
*After initial attention scores computed, they are scaled and transformed nonlinearly by applying a softmax

Value transformation

#	decoded token
1	Hydrange
2	as
3	,
4	Pale
5	blue
6	in
7	the
8	rain
9	,
10	Blue
11	in
12	the
13	moonlight

#	decoded token
1	Hydrange
2	as
3	,
4	Pale
5	blue
6	in
7	the
8	rain
9	,
10	Blue
11	in
12	the
13	moonlight

Per column Softmax (over rows), downscaling and rewriting to array

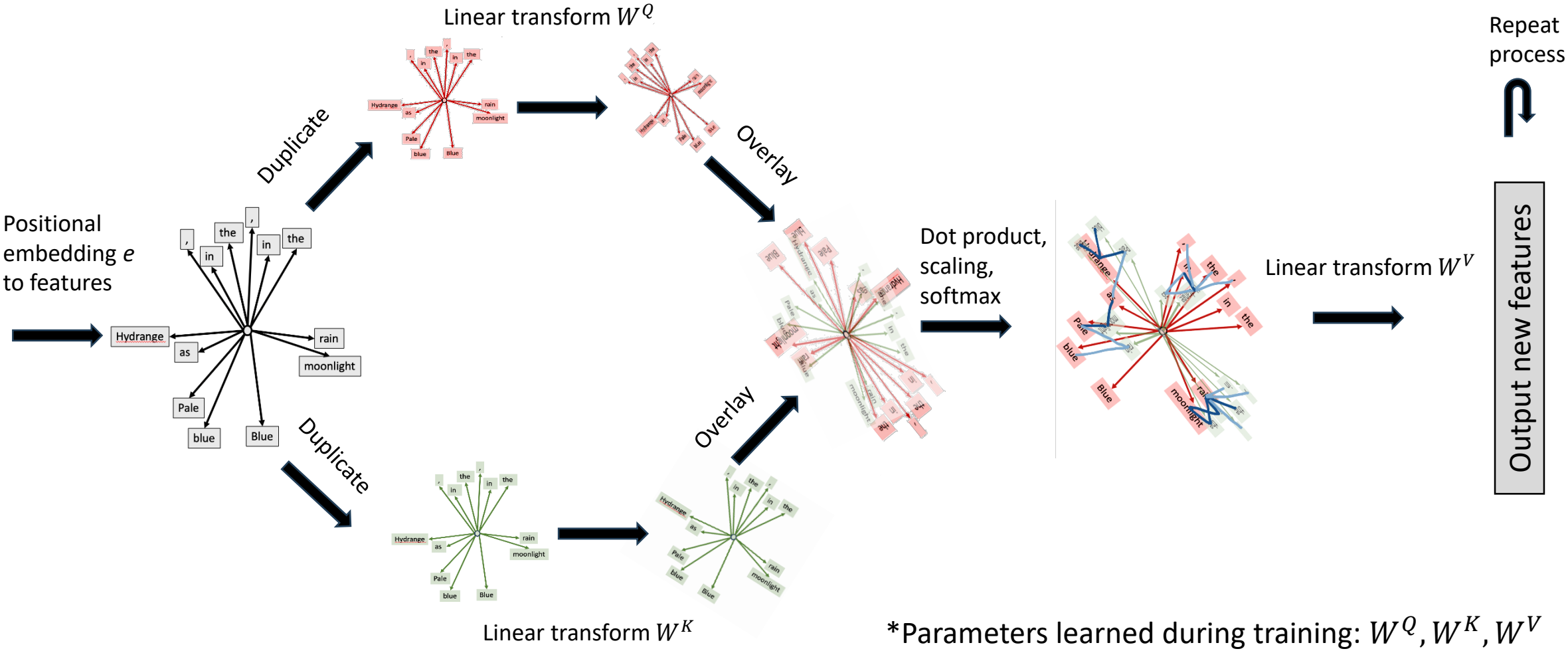


Linear transform W_V

New "Representation"

Summary of Attention

Hydrangeas, Pale blue in the rain, Blue in the moonlight



*Parameters learned during training: W^Q, W^K, W^V

Transformer

What we just outlined – single-head attention – is limited

- intuitively, can learn "one kind of attention"

Rather than just one attention layer, use *multi-headed attention*

- have multiple attention mechanisms (with their own parameters) running together
- allows learning multiple "representations" of context – e.g.,
  - grammar
  - meaning
  - position

Transformer refers to a DNN architecture that

1. stacks attention mechanisms
2. combines with more traditional NN structures

4. Anatomy of an LLM

Next Token Prediction

Basic Task: Predict the Next Token

A simple way to describe what an LLM is trained to do:

- Given text so far (*context*), predict what comes next
- Example: Complete the Haiku fragment

  Hydrangeas, Pale blue in the _____

- First
  - Create a probabilistic model for next token: Pr(Next token = Token j|Context)
    - E.g. Need a model for Pr(Next token = rain | Hydrangeas, Pale blue in the)
    - Conceptually - a (big) supervised learning problem (multi-class classification)
- Then
  1. Draw new token according to model
  2. Append token to phrase and repeat

This is a big "model." Lots of tokens (10s of thousands) and **LOTS** of contexts.

We're building all of this off the previous structure and relying on many contexts being "close."

Training: Masked Token Prediction

We can manufacture training data by *masking* raw text

- From any document, get many training examples.
  - E.g.,
    - Hydrangeas, [MASK] blue in the rain, Blue in the moonlight
    - Hydrangeas, Pale [MASK] in the rain, Blue in the moonlight
    - Hydrangeas, Pale blue [MASK] the rain, Blue in the moonlight
    - Hydrangeas, Pale blue in [MASK] rain, Blue in the moonlight
    - Hydrangeas, Pale blue in the [MASK], Blue in the moonlight

Different approaches will treat light gray context differently

- Use remaining context to build model for predicting masked token
  - Choose token specific scores $\left(f_1(X), \ldots, f_J(X)\right)$ for context X to minimize loss
    - $f_j(X)$ is a number associated with output token j given context X where a higher number indicates token j is more likely

E.g. cross-entropy

Aside: Encoders and Decoders

Many ways to build a language model

- Encoders (e.g. BERT from Google)
  - Designed to build representation of text
  - Can look at all tokens at once (left and right of context)
  - Trained with masked token prediction
  - Best for classification, search, similarity, …
- Decoders (e.g. GPTn from OpenAI)
  - Designed to generate text
  - Can only look at previous tokens
  - Trained with next token prediction
  - Best for generation, completion, writing, …
- Encoder-Decoder (e.g. T5 from Google)
  - Combines both
  - Encoder reads, decoder generates
  - Best for translation, summarization, rewriting, …

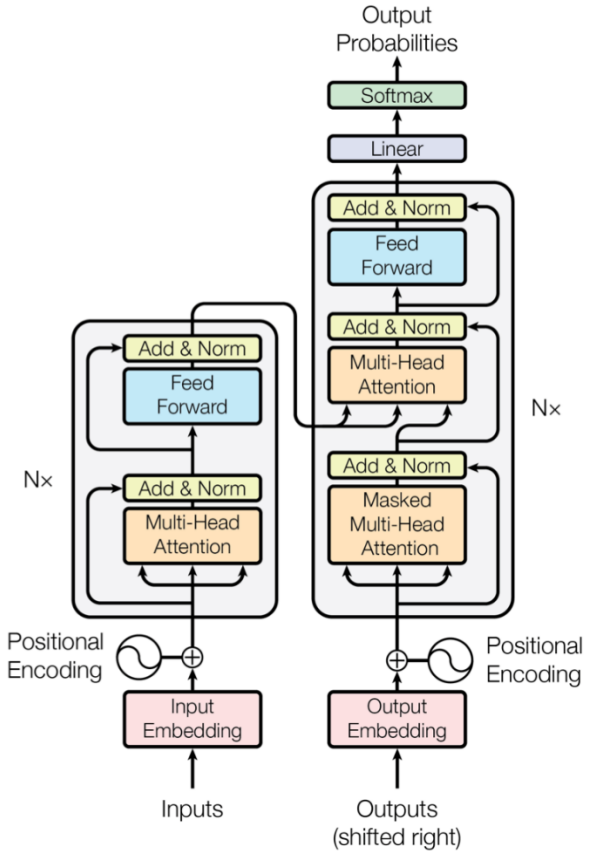


Figure 1: The Transformer - model architecture.

From Scores to Probabilities

We have

- X: current context
- $f_j(X)$: score from model for j-th token (for $j = 1, \ldots, J$)

Want to generate next token (Y)

We turn what we have into a token probability by applying the softmax with a temperature (T, a tuning parameter)

$$\Pr(Y = token\ j|X) = \frac{\exp\left(\frac{f_j(X)}{T}\right)}{\sum_{q=1}^{J} \exp\left(\frac{f_q(X)}{T}\right)}$$

- Large T flattens probabilities, low-ranked tokens become more likely
- Small T sharpens probabilities, most likely tokens dominate (becomes deterministic as $T \to 0$)
- For many current commercial LLMs, decoding parameters (including temperature) are partially or fully managed by the system – i.e. not available to play/experiment with for most users

Example: Next Token for Our Haiku

Suppose we want to fill in the next word

We're use GPT-4.1-mini for this example. I've cheated a bit and also provided some instructions so it actually tries to finish the statement rather than its default behavior which is to try to chat.

Hydrangeas, Pale blue in the _____

Context

Top 5 "most likely" completions

Token	Temp = 0.1	Temp = 0.2	Temp = 0.7	Temp = 1.0	Temp = 1.5
mor	1	0.988	0.763	0.655	0.513
Hyd	0	0.012	0.219	0.273	0.286
early	0	0	0.011	0.033	0.069
soft	0	0	0.004	0.015	0.042
garden	0	0	0.003	0.014	0.039

Due to randomness in API calls, these numbers will change across instances.

Repeat the Process

- Hydrangeas, Pale blue in the _____
  - mor (.6070)
  - Hyd (.3249)
  - garden (.0267)
  - early (.0208)
  - m (.0060)
- Hydrangeas, Pale blue in the mor_____
  - ning (.9824)
  - nings (.0159)
  - Hyd (.0017)
  - ing (.0000)
  - n (.0000)
- Hydrangeas, Pale blue in the morning_____
  - Hyd (.9750)
  - and (.0139)
  - hyd (.0031)
  - soft (.0013)
  - turn (.0011)

Due to randomness in API calls, these numbers will change across instances.

What's happening here?

If you keep filling in token by token from this point, you get a loop of 'Hyd', 'Hyd', …

There's a lot going on in decoding beyond the "model" that is hidden

RLHF – Training the Model to "Behave"

- Pretraining (what we just talked about)
  - Learn to predict next token from massive collection of data
  - Objective: *statistical accuracy*

- Human Feedback
  - Humans compare model answers
    - E.g. Response A is better than Response B
      - You've likely seen this if used any tool for even a modest amount of time
  - Trains a reward model that predicts user preferences

- RL
  - Model *refined* to maximize reward model score
    - not focused entirely on statistical accuracy (though still matters)
  - Encourages helpful answers, following instructions, avoiding unsafe/harmful content, …

Decoding is More than Sampling

LLM outputs further shaped by controls layered on top of token probabilities

- Anti-Repetition Controls
  - Penalize repeating the same token
    - prevents loops (like “Hyd”, “Hyd”, … noted above)
  - often called frequency or presence penalty
- Top-k and Top-p (Nucleus) Sampling
  - Don’t sample from all tokens
  - Only consider most plausible
  - Helps keep output coherent while allowing variation via sampling
- Safety and Policy Filters
  - System blocks or steers away from unsafe content
  - Happens both during and after generation
- Instruction Tuning
  - Model explicitly tuned to follow prompts like
    - “Explain…”
    - “Summarize…”
    - “Answer as an expert…”
  - Pushes the model into more helpful answers, not just random text completer

Final output reflects:

training + human feedback + decoding rules

Much more than model based token frequencies

Looking at the Process Using “Hidden Tricks”

- Hydrangeas, Pale blue in the _____
  - mor (.5916)
  - Hyd (.3588)
  - early (.0202)
  - soft (.0108)
  - garden (.0108)

Here, we’re not generating one token at a time but having the LLM generate many tokens using all the decoding tricks layered into the background.

- Hydrangeas, Pale blue in the morning light, bloom softly along the garden path. Their delicate petals, ting_____
  - ed (.9999)
  - ing (.0001)
  - The (.0000)
  - ted (.0000)
  - ered (.0000)

Due to randomness in API calls, these numbers will change across instances.

5. Few-Shot Learning

Adapting Foundation Models

Foundation models are general purpose and potentially useful for many tasks

BUT they are not purpose built for YOUR task

Adapting models to specific tasks can offer real benefits

- Use of internal, firm-specific knowledge
  - Modest amounts of high-quality, specific data can trump large amounts of general data
- Better performance on task of interest
- Possible reduction in model complexity (“small LMs”) via distillation
  - E.g., use LLM as “teacher” to train small “student” model for specific task

Fine-Tuning

One option is fine-tuning

- Use your own labeled data to update the model's parameters
- Recall we discussed this in our DNN lecture earlier
- Relatively lightweight versions available (e.g. LoRA – low-rank adaptation)

Pros

- Carefully tailored (via training data) to specific task of interest
- Good for high-volume, stable tasks
- "Own" the model. Can look more carefully at governance, safeguards, …

Typically fine-tuned from other provider's model via an API so don't technically own.

Cons

- Requires data, ML expertise, infrastructure
- Slower development cycles
- Cost (development, data, maintenance, …)

Few-Shot Learning

Foundation models offer an interesting alternative to fine-tuning

Few-Shot Learning

- Don’t explicitly update model parameters
- Provide examples of desired behavior (input-output examples) as part of the context
- Model infers desired pattern from prompt and applies to new cases

Essentially, fine-tuning without training

Pros

- Fast and lightweight
- No modeling or infrastructure
- No deep knowledge of ML necessary

Cons

- Typically relies on external model and long prompts (costs add up)
- Harder to control “unexpected behavior” if deployed at scale
- Limited examples can lead to poor “generalization”

Example: Resume Router

Suppose you wanted an AI to “read” submitted resumes in an effort to route them to the correct department

We’ll use OpenAI’s GPT-4.1-mini as our base model

Pipeline:

- Read first 150 tokens in resume
- Assign to appropriate team
  - 11 teams
- Compare base model performance (zero-shot) to model with few-shot learning prompt
  - 6 examples per team for few shot learning
  - Have 2418 resumes held out for evaluation

Base Model

We pass simple instructions to base model:

```
You are routing resumes to the correct department.
Return EXACTLY ONE LINE in this format:
DEPARTMENT=<LABEL>;CONF=<NUMBER>
Where <LABEL> is exactly one of the allowed
labels, and <NUMBER> is a decimal in [0,1].
The line must begin with DEPARTMENT=.
No extra text. No markdown. No code fences.
Allowed labels: [{labels}]
```

- `labels` is a variable containing the department names

We then pass each resume snippet in a prompt:

```
Resume:
       -----
       {resume_snippet}
       -----
       Classify this resume into the correct
department.
```

- `resume_snippet` is a variable containing the resume text

Instructions and prompt important for performance. (TBH, I didn't spend much time on them.)

Accuracy of base model: 53.6%

Code for this example is "FewShotIllustration.py"

Few-Shot Learning

We use almost the same instructions:

```
You are routing resumes to the correct department.
Use the examples to infer the routing standard.
Return EXACTLY ONE LINE in this format:
DEPARTMENT=<LABEL>;CONF=<NUMBER>
Where <LABEL> is exactly one of the allowed
labels, and <NUMBER> is a decimal in [0,1].
The line must begin with DEPARTMENT=.
No extra text. No markdown. No code fences.
Allowed labels: [{labels}]
```

- `labels` is a variable containing the department names

Accuracy with few shot learning: 61.2%

Significantly (statistically) superior to base performance

FWIW, this example cost me $31.52 (in early February 2026) to prototype and run

- includes several runs to debug and check stability
- Approx. $0.50 for base model, rest for few-shot learning
- Is the 8 percentage point improvement in accuracy worth it?

Our prompt is MUCH longer:

```
Here are labeled examples:
        join(ex_blocks)
Now classify this new resume. Return exactly
one line in the required format:
        -----
        {resume_snippet}
        -----
```

- `resume_snippet` contains the text to be classified
- `ex_blocks` is a block of examples giving resume text AND correct department (~66=(6 examples)*(11 teams) times longer than original prompt)

```
Example {k} Resume:
     -----
     {ex['resume_snippet']}
     -----
     Correct department:{ex['department']}
```

6. Agents

Building from Foundation Models

Foundation models, e.g. LLMs:

- built from massive, diverse data
- meant to be general purpose – adaptable to many tasks
- can serve as a "brain" in complex tasks

Combining foundation models and *tools* allows automation of many tasks

- Tools
  - e.g. 1) web search, 2) write and execute code, 3) access database, 4) use a calculator, …

Agent: foundation model combined with tools that can

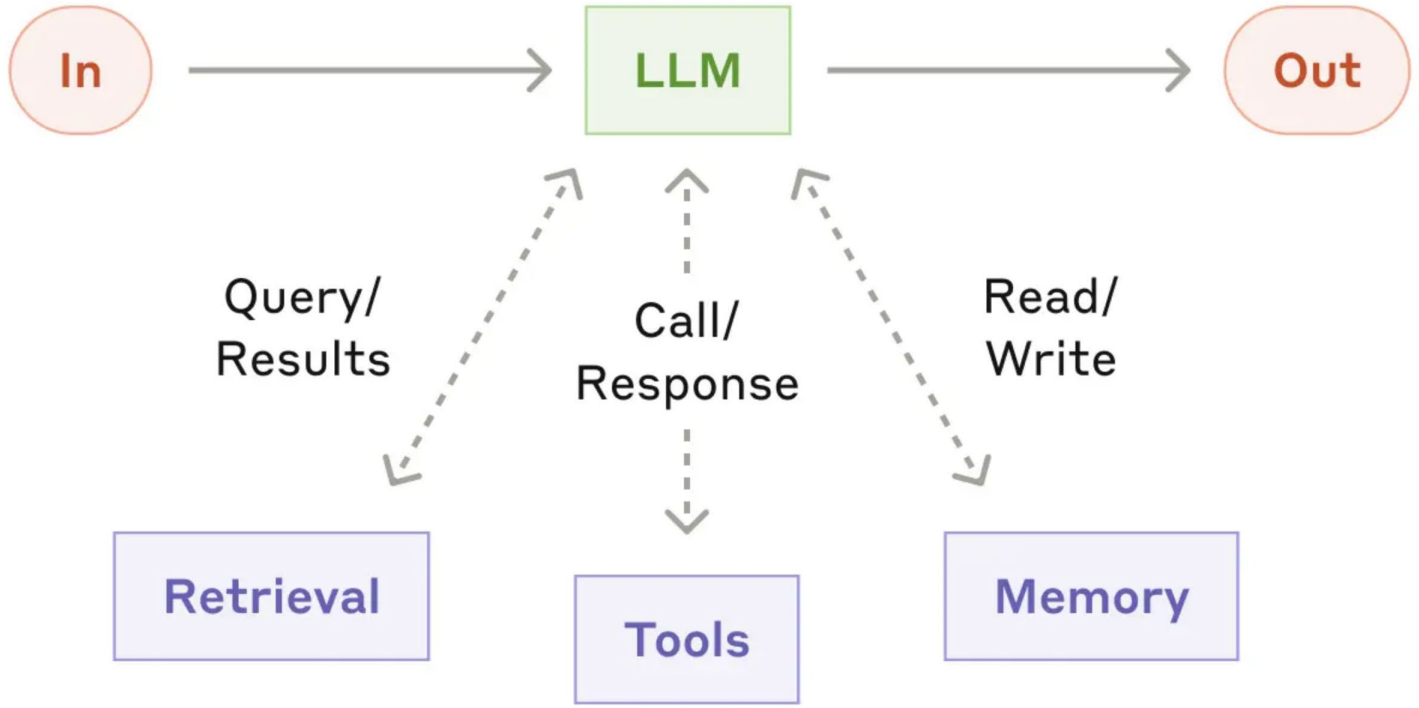
- direct own tool usage
- learn from the environment
- adapt and attempt to solve problems

https://www.anthropic.com/research/building-effective-agents

Augmented LLM

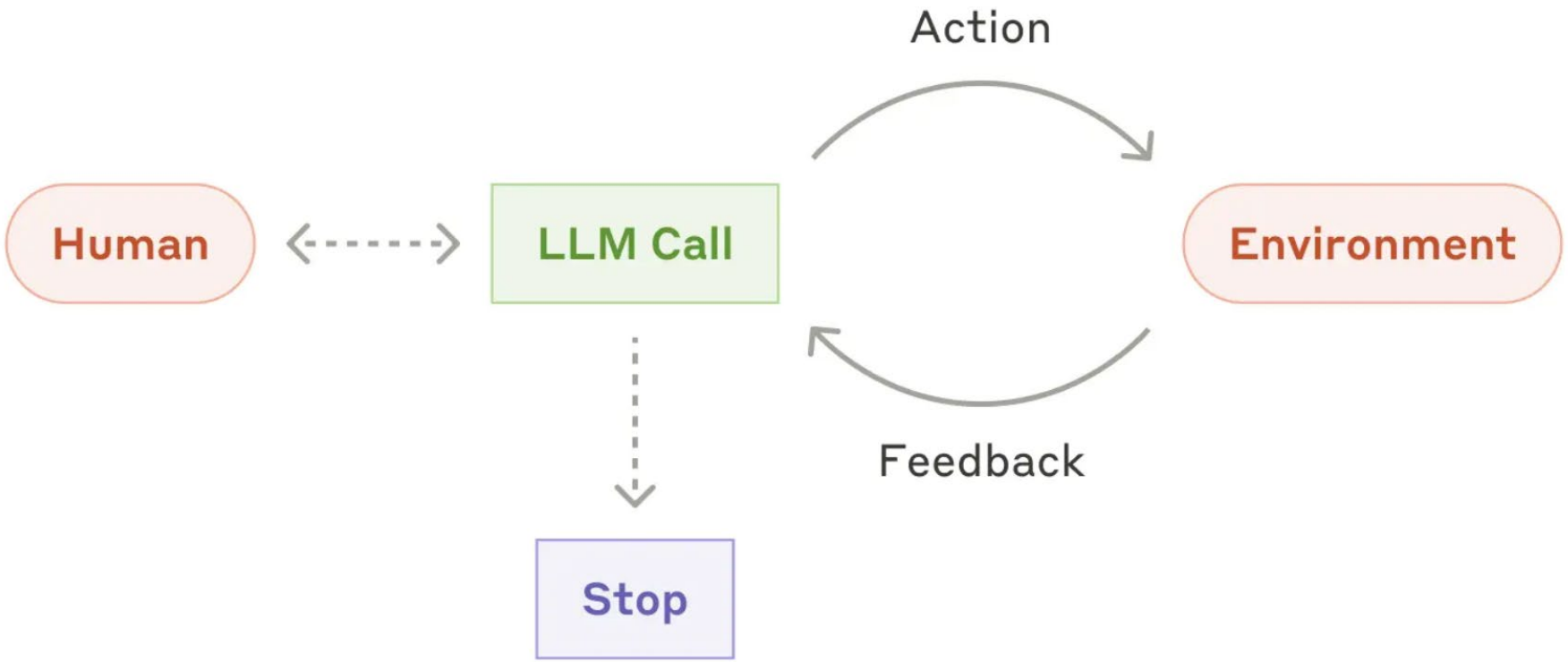
Basic building block:

Augmented foundation model



https://www.anthropic.com/research/building-effective-agents

Example Agent



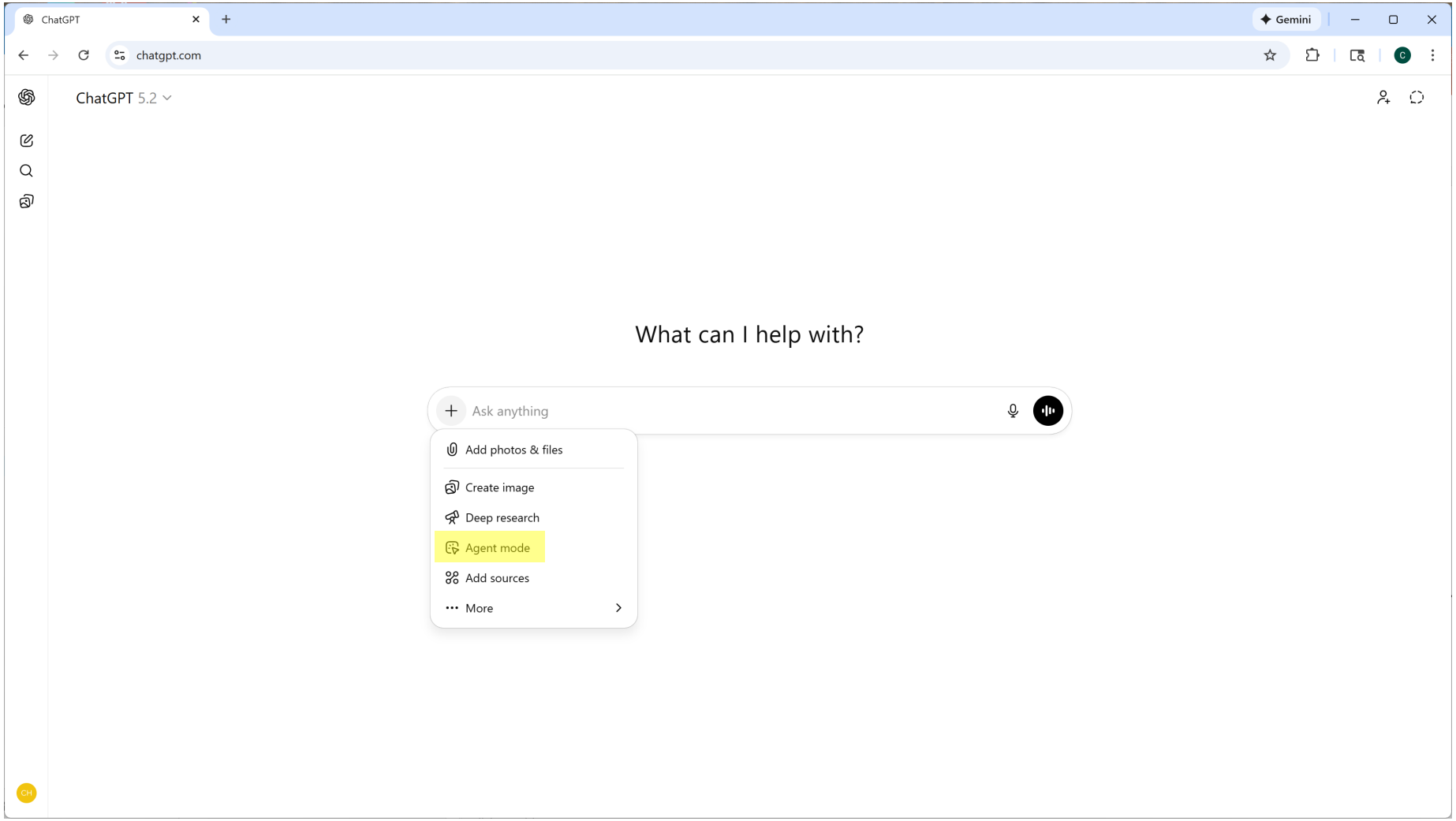
https://www.anthropic.com/research/building-effective-agents

Agentic AI

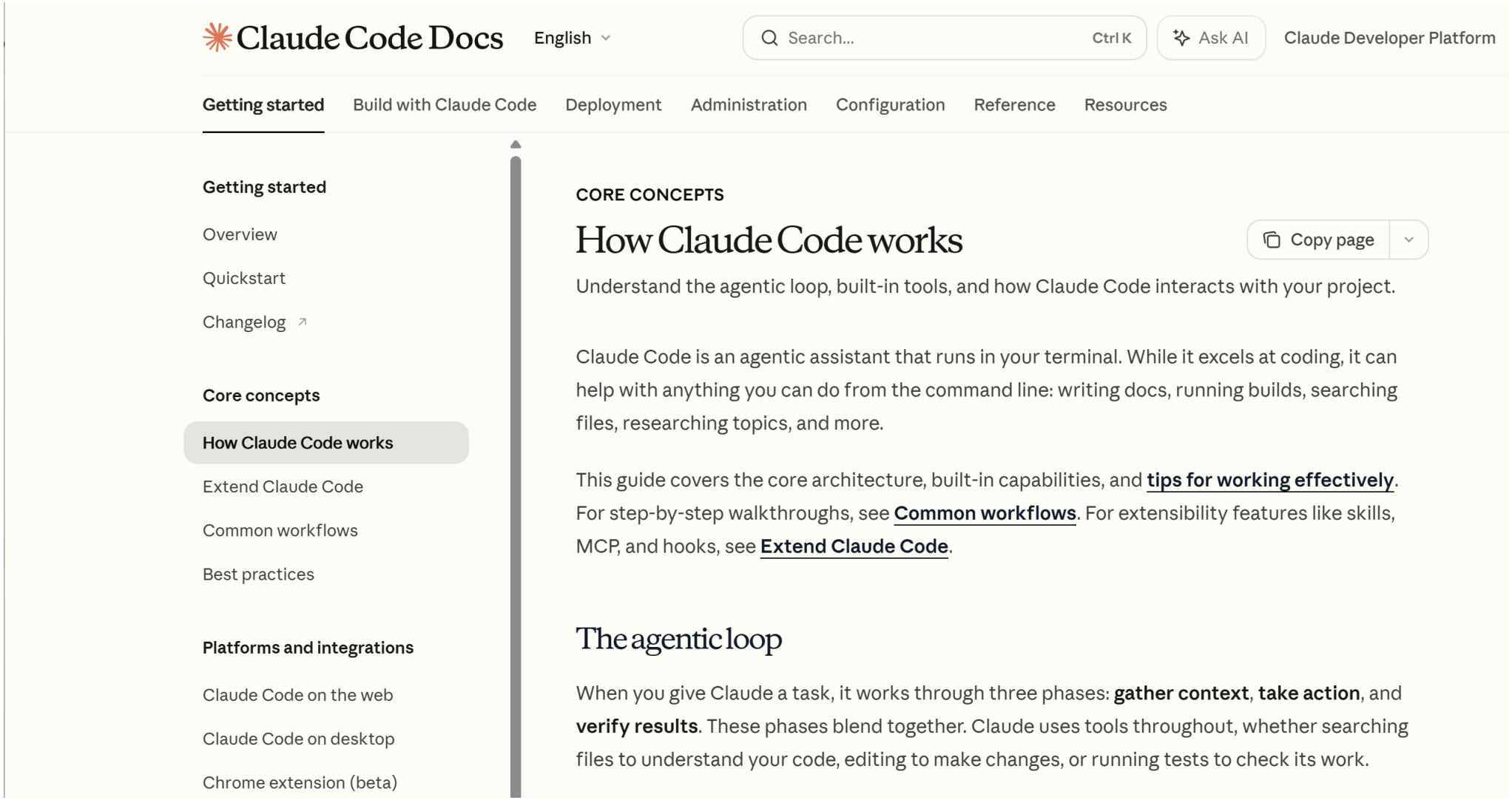
Three (somewhat fuzzy) categories:

- Workflow – foundation model with tools that follows prescribed tasks
- Agent – foundation model with tools that can perceive environment, make decisions, and respond to environment
- Agentic AI – agent or group of interacting agents (and humans) capable of perceiving environment, long-term decision-making, adaptation, coordination with other agents, self-improvement

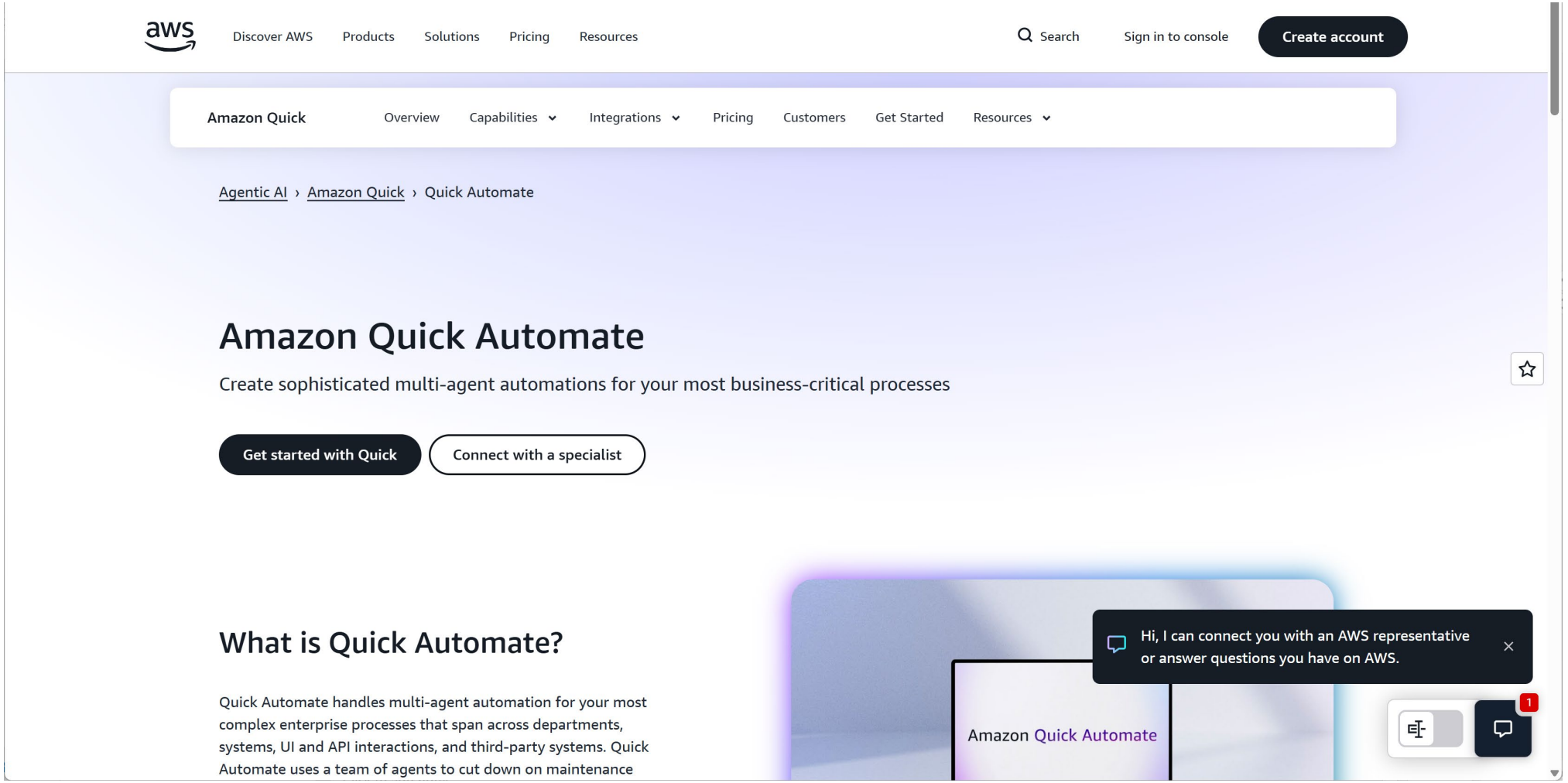
Agent Example



Agent Example



Agent Example



Basic Agentic RAG

A simple example of agentic AI is *Agentic RAG* (Retrieval-Augmented Generation)

- RAG connects a database to a LLM (outside of its training data) so that it can reference that information while generating responses
  - provides domain-specific and up-to-date information
  - e.g. give access to internal company data/documents

- Our simple example will have one agent with a single tool:
  - Database with current draft of two chapters from Applied Causal Inference Powered by ML and AI
  - Agent will choose whether to use the tool or default (GPT-4.1-mini) response completion
  - Tool implemented by building custom tool in Python
    - Many prebuilt tools, agents, connectors out there in the wild

- Agentic RAG can incorporate additional tools
  - Just using one as illustration

Prompt for Our RAG Agent

We use an LLM with custom instructions as the “brain” of our agent

Need to help agent “think” about using the tool

```
You are a helpful assistant that can answer questions on any topic.

You have an optional tool:
- search_book(query, k): returns relevant excerpts from a causal inference / causal ML textbook with chapter + page ranges.

Decision policy (follow strictly):
- If the question is likely answerable from the causal inference textbook …, then call search_book first.
- If the question is clearly unrelated …, do not call the tool.
- If you are uncertain whether it relates to causal inference, do one quick search_book call with a short query anyway, and
use it only if results are relevant.

How to answer:
1) Start with a brief one-line “plan” only when helpful (e.g., “I’ll check the book for how it frames cross-fitting, then
give a template.”). If you did not use the tool, do not mention it.
2) If you used search_book and results are relevant: ground the answer in those excerpts and cite “Ch X, pp. a–b” for key
steps/claims.
3) If you used search_book but results are not relevant: say the excerpts didn’t directly address it, then answer from
general knowledge.
4) Never refuse to answer just because the book doesn’t cover it.

Output format (only when the question is causal-methods related):
RECOMMENDATION
ASSUMPTIONS / CHECKS
PYTHON TEMPLATE
CITED SUPPORT (bullets with Ch, pages)

Otherwise, respond in the most natural format for the question.
Tone: clear, practical, not overly formal.
```

Agentic RAG

Prompt

I have outcome Y, treatment D, and high-dimensional controls X under unconfoundedness. Recommend an approach and give a Python template with cross-fitting, and cite the book.

RECOMMENDATION

Under unconfoundedness with high-dimensional controls X, a solid approach is to use Double Machine Learning (DML) with cross-fitting. This method estimates the nuisance functions (outcome regression and propensity score) using flexible machine learning models on one part of the sample, then uses these estimates to construct orthogonalized moment conditions on another part to remove bias from overfitting. Cross-fitting guards against overfitting bias and improves inference validity.

ASSUMPTIONS / CHECKS

- Unconfoundedness: Treatment D is independent of potential outcomes conditional on X.
- Overlap: Each unit has a positive probability of receiving both treatments conditional on X.

⋮

CITED SUPPORT

- Double Machine Learning uses cross-fitting to remove "own-observation bias" from estimating nuisance parameters (Ch 9, pp. 3, 5, 11).
- Residualizing Y and D with estimated nuisance functions based on machine learning avoids overfitting bias (Ch 9, pp. 5).
- Cross-fitting is a critical step to robustify against overfitting when using flexible ML methods like RF, boosting, etc. (Ch 9, pp. 3–6).

Output from GPT 4.1-mini with RAG (2/11/26)

Given your setup: outcome (Y), treatment (D), and high-dimensional controls (X) with the assumption of unconfoundedness (i.e., conditional independence (Y(d) \perp D \mid X)), a recommended approach for estimating the treatment effect is the **Double/Debiased Machine Learning (DML)** framework using cross-fitting.

This approach is well covered in the book:

Chernozhukov, Victor, et al. (2018). "Double/debiased machine learning for treatment and structural parameters." The Econometrics Journal, 21(1), C1–C68.

and summarized in the book:

Chernozhukov, Victor, et al. (2022). "Double/Debiased Machine Learning." Cambridge University Press.

Plausible sounding but not a real book

⋮

Output from GPT-4.1-mini (2/11/26)

Agentic RAG

Prompt

What are the steps to DML?

More common meaning of DML but not what book is about

The Double Machine Learning (DML) procedure follows these general steps:

1. Define a Neyman orthogonal score function for the target low-dimensional parameter of interest (e.g., treatment effect). This score function depends on the data and nuisance parameters like propensity scores and outcome regressions.
2. Split the data into K folds for cross-fitting. For each fold:
   - Estimate the nuisance parameters (e.g., propensity score, outcome regression) using machine learning methods on the data excluding this fold.
   - Use these estimated nuisance parameters to compute the orthogonal score on the held-out fold.
3. Combine the orthogonal scores across folds to form an empirical moment condition.

Output from GPT 4.1-mini with RAG (2/11/26)

The term "DML" usually stands for **Data Manipulation Language** in the context of databases. DML refers to the set of SQL commands used to retrieve and manipulate data within database tables.

If your question is about the **steps to perform Data Manipulation Language operations**, here are the basic steps typically involved:

1. **Connect to the Database**
   Establish a connection to the database where you want to perform operations.
2. **Select the Target Table**
   Identify which table(s) you wish to manipulate data in.
3. **Perform DML Operations**
   The common DML commands are:

Output from GPT-4.1-mini (2/11/26)

Agentic RAG

Prompt

I'd like a recipe for paella valenciana.

Not related to book, so doesn't use tool.

Sure! Here's a classic recipe for Paella Valenciana, a traditional Spanish dish from the Valencia region:

Ingredients:

- 2 cups bomba or short-grain rice
- 1/4 cup olive oil
- 1 onion, finely chopped
- 1 tomato, grated or finely chopped
- 1 red bell pepper, sliced
- 2 cloves garlic, minced
- 1/2 lb chicken, cut into pieces
- 1/2 lb rabbit (optional), cut into pieces
- 1/2 cup green beans (like flat green beans or romano beans)
- 1/2 cup garrofón (large white beans) or lima beans
- 4 cups chicken broth
- Saffron threads or yellow food coloring (for color and flavor)
- Salt, to taste
- Fresh rosemary (whole sprig)
- Lemon wedges, for serving

Instructions:

⋮

Output from GPT 4.1-mini with RAG (2/11/26)

Certainly! Here's a traditional recipe for **Paella Valenciana**, a classic Spanish rice dish from the Valencia region.

Ingredients:

- 1/4 cup olive oil
- 1 whole chicken (cut into pieces)
- 1 whole rabbit (cut into pieces) – optional, you can substitute with more chicken or omit
- 200g green beans (ferraura or similar)
- 100g runner beans (optional)
- 2 ripe tomatoes, grated or finely chopped
- 100g garrofón (large white beans) or lima beans
- 1 red bell pepper (optional)
- 2 cups Bomba or short-grain Spanish rice
- 4 cups chicken broth (preferably homemade or low sodium)
- 1 pinch saffron threads (soaked in a little warm water)
- 1 teaspoon sweet smoked paprika
- Salt to taste
- Fresh rosemary (optional)
- Lemon wedges for serving

Instructions:

⋮

Output from GPT-4.1-mini (2/11/26)

Summary

Foundation models can serve as “brains” in AI systems and help AI perceive and interact with environment

- E.g. LLMs to understand human language, instructions, and communicate with users
- E.g. image processing models for self-driving cars, robot warehouse workers

Agents offer possibility of autonomous AI systems that can perform tasks and make decisions

Current “hot topic” in AI with many potential applications